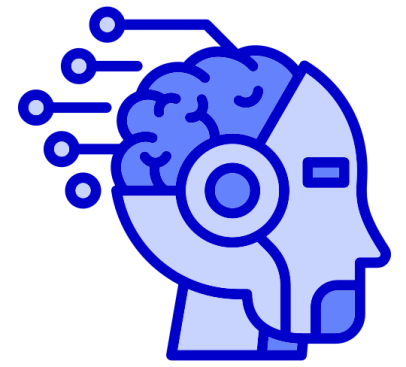




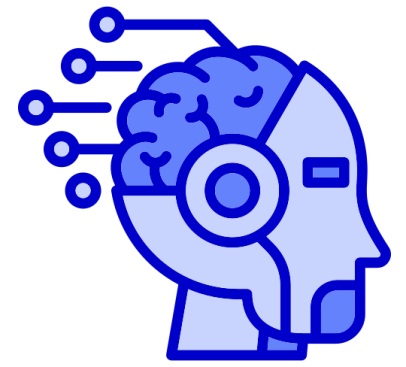
Uninformed Searches

IDS

Lecture 6

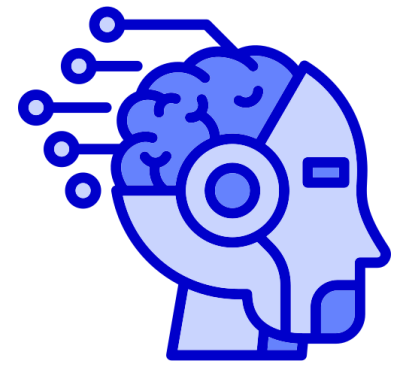
Artificial Intelligence

Properties of Uniform-Cost Search



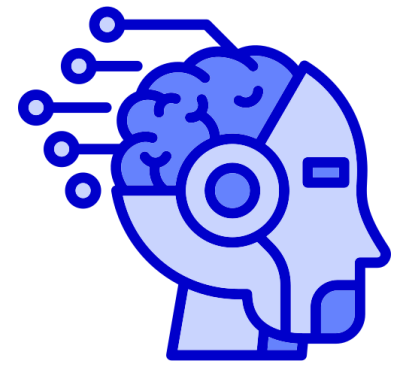
- **Complete:** Yes if arc costs > 0 .
- **Optimal:** Yes
- Time and space complexity: $O(b^d)$ (i.e., exponential)
 - d is the depth of the solution
 - b is the branching factor at each non-leaf node
- More precisely, time and space complexity is $O(b^{C^*/\epsilon})$ where ϵ is the minimum edge cost $\epsilon > 0$, and C^* is the best goal path cost

Iterative-Deepening Search(IDS)



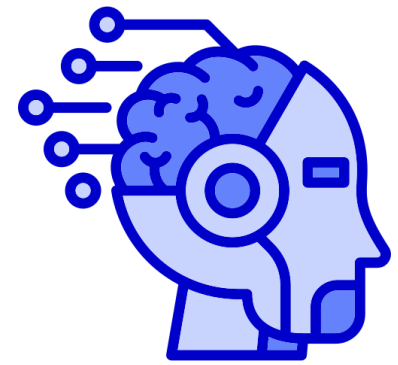
- requires modification to DFS search algorithm:
 - do DFS to depth 1
and treat all children of the start node as leaves
 - if no solution is found, do DFS to depth 2
 - repeat by increasing “depth bound” until a solution found
- Start node is at depth 0

Iterative-Deepening Search(IDS)



```
Node deepeningSearch (Problem problem) { int depth
    = 1;
    Stack DSnodes = new Stack();
    while (true) {    // while not solved
        Node node = DFS_depthBound(problem, DSnodes, depth);
        // DFS_depthBound limits DFS search to level <= depth
        if (node isn't "failure") return node; // solved
        depth++; // look deeper
    }
}
```

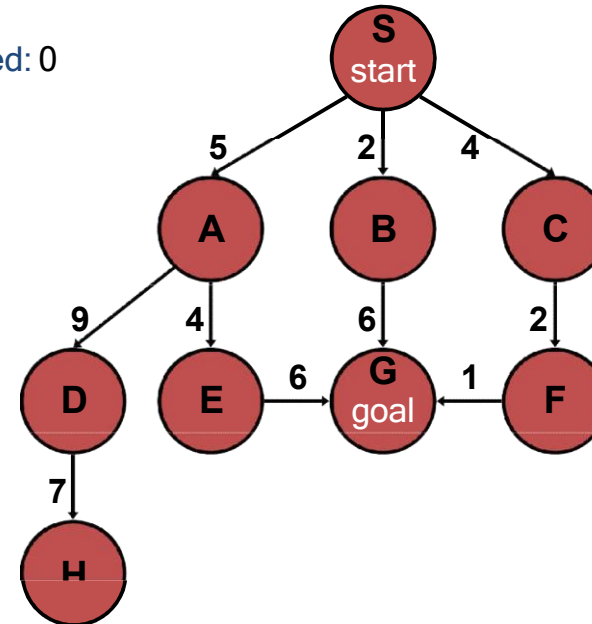
Iterative-Deepening Search(IDS)



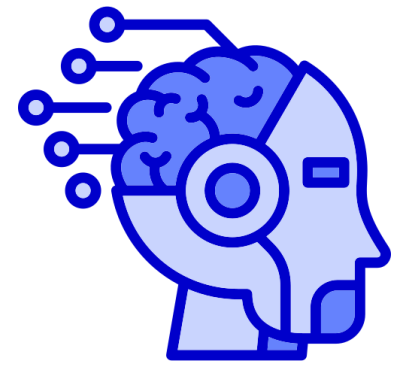
deepeningSearch (problem)

depth: 1, # of nodes expanded: 0, tested: 0

expnd. node	Frontier
	{S}



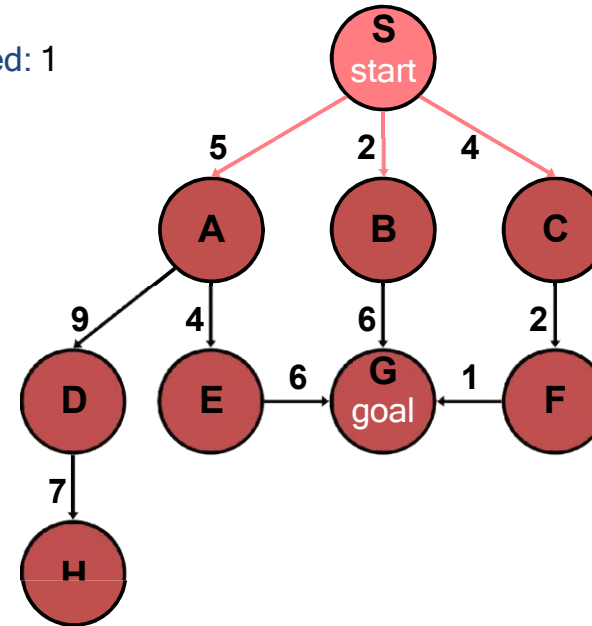
Iterative-Deepening Search(IDS)



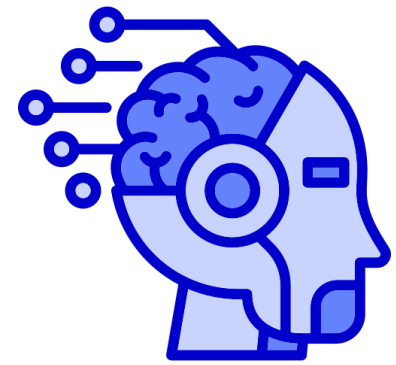
deepeningSearch (problem)

depth: 1, # of nodes tested: 1, expanded: 1

expnd. node	Frontier
	{S}
S not goal	{A,B,C}



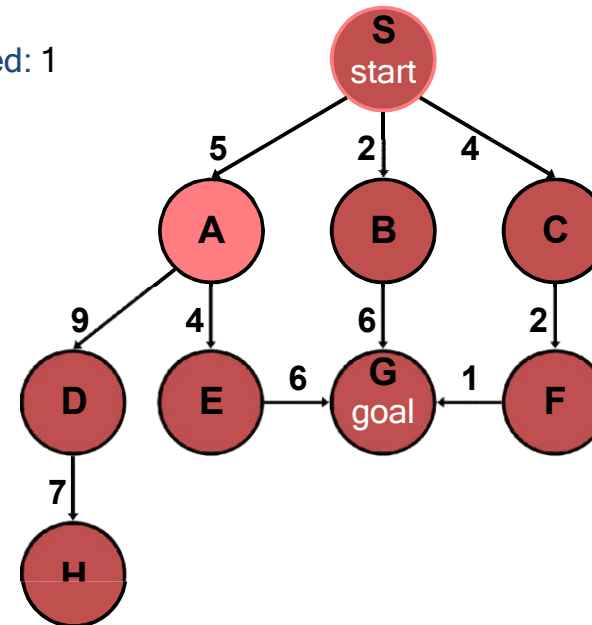
Iterative-Deepening Search(IDS)



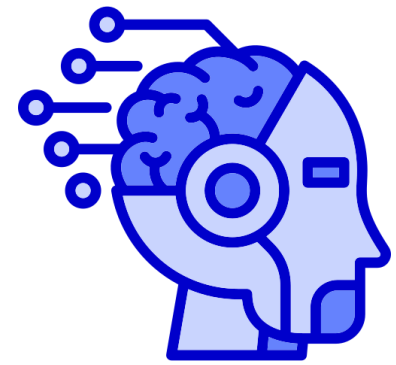
deepeningSearch (problem)

depth: 1, # of nodes tested: 2, expanded: 1

expnd. node	Frontier
	{S}
S	{A,B,C}
A not goal	{B,C} no expand



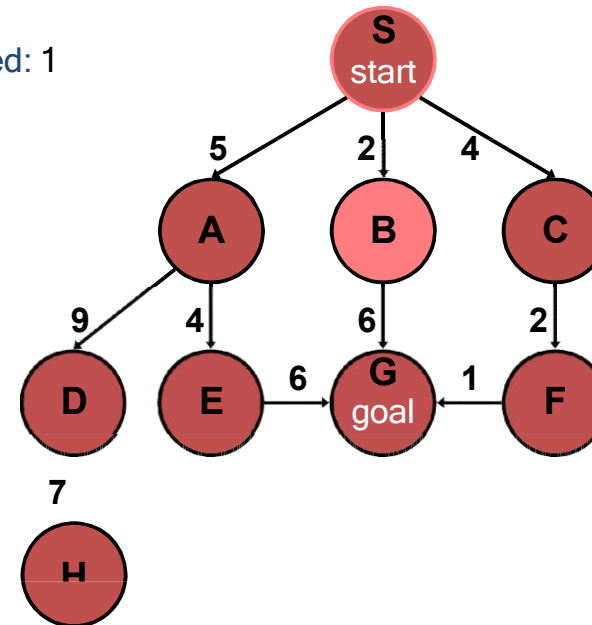
Iterative-Deepening Search(IDS)



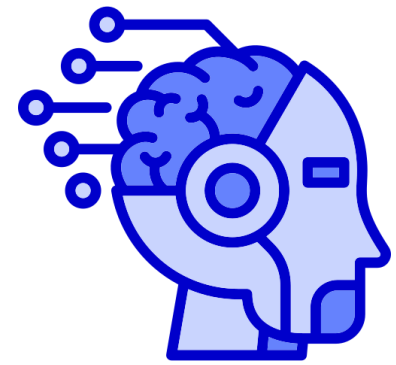
deepeningSearch (problem)

depth: 1, # of nodes tested: 3, expanded: 1

expnd. node	Frontier
	{S}
S	{A,B,C}
A	{B,C}
B not goal	{C} no expand



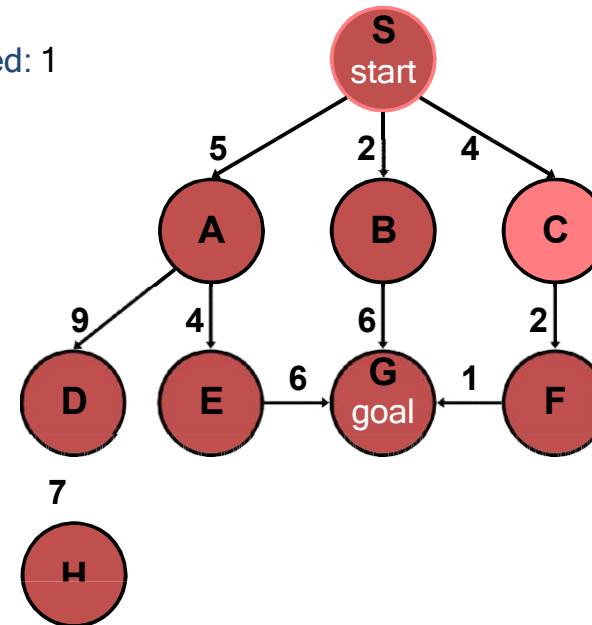
Iterative-Deepening Search(IDS)



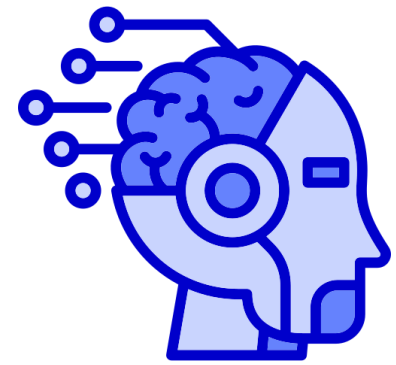
deepeningSearch (problem)

depth: 1, # of nodes tested: 4, expanded: 1

expnd. node	Frontier
	{S}
S	{A,B,C}
A	{B,C}
B	{C}
C not goal	{ } no expand-FAIL



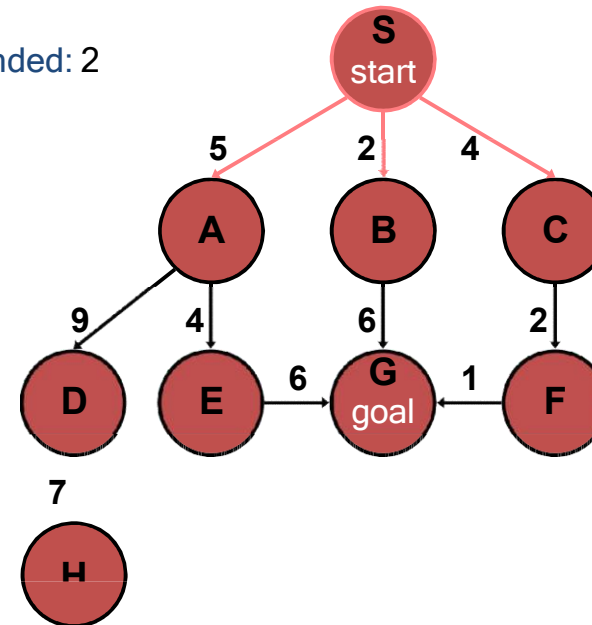
Iterative-Deepening Search(IDS)



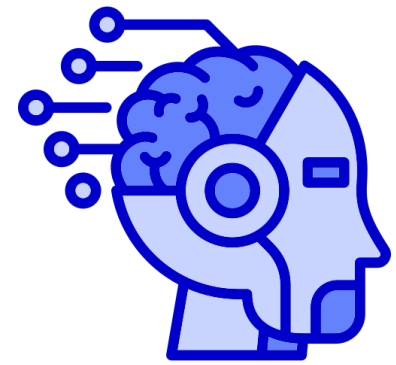
deepeningSearch (problem)

depth: 2, # of nodes tested: 4(1), expanded: 2

expnd. node	Frontier
	{S}
S	{A,B,C}
A	{B,C}
B	{C}
C	{}
S no test	{A,B,C}



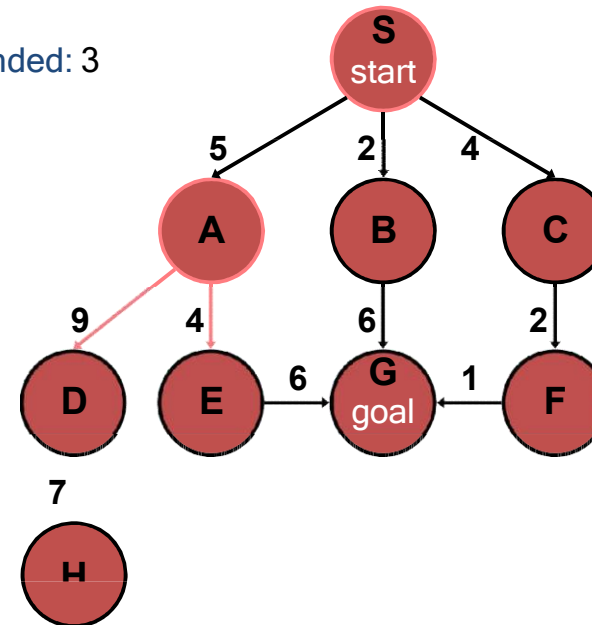
Iterative-Deepening Search(IDS)



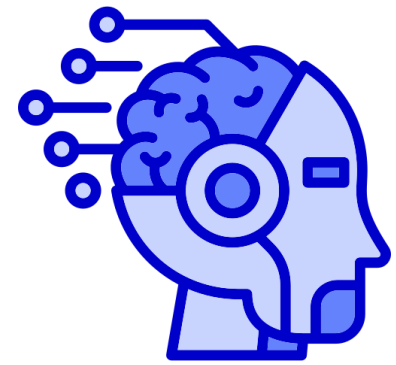
deepeningSearch (problem)

depth: 2, # of nodes tested: 4(2), expanded: 3

expnd. node	Frontier
	{S}
S	{A,B,C}
A	{B,C}
B	{C}
C	{}
S	{A,B,C}
A no test	{D,E,B,C}



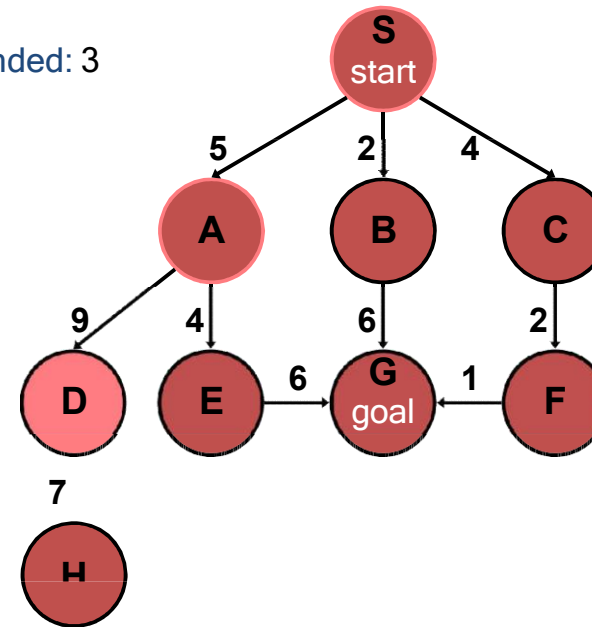
Iterative-Deepening Search(IDS)



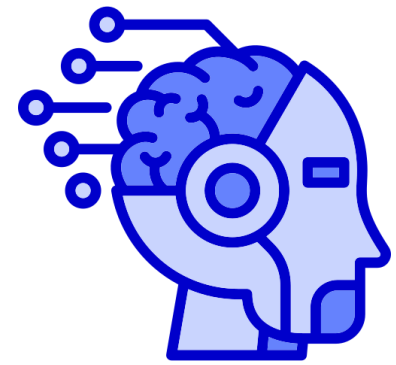
deepeningSearch (problem)

depth: 2, # of nodes tested: 5(2), expanded: 3

expnd. node	Frontier
	{S}
S	{A,B,C}
A	{B,C}
B	{C}
C	{ }
S	{A,B,C}
A	{D,E,B,C}
D not goal	{E,B,C} no expand



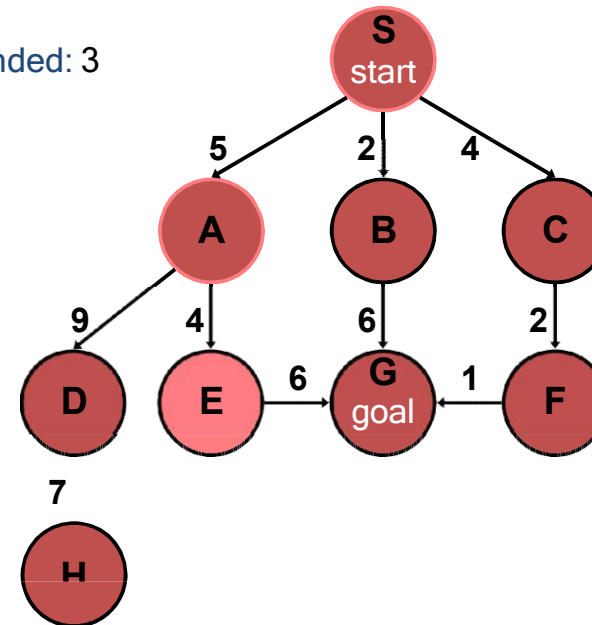
Iterative-Deepening Search(IDS)



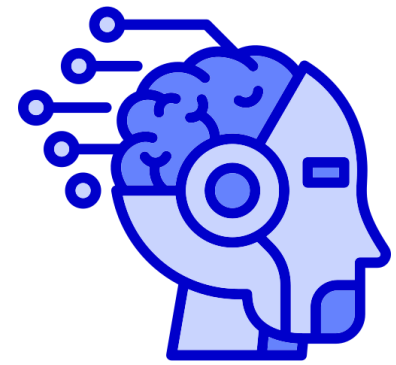
deepeningSearch (problem)

depth: 2, # of nodes tested: 6(2), expanded: 3

expnd. node	Frontier
	{S}
S	{A,B,C}
A	{B,C}
B	{C}
C	{ }
S	{A,B,C}
A	{D,E,B,C}
D	{E,B,C}
E not goal	{B,C} no expand



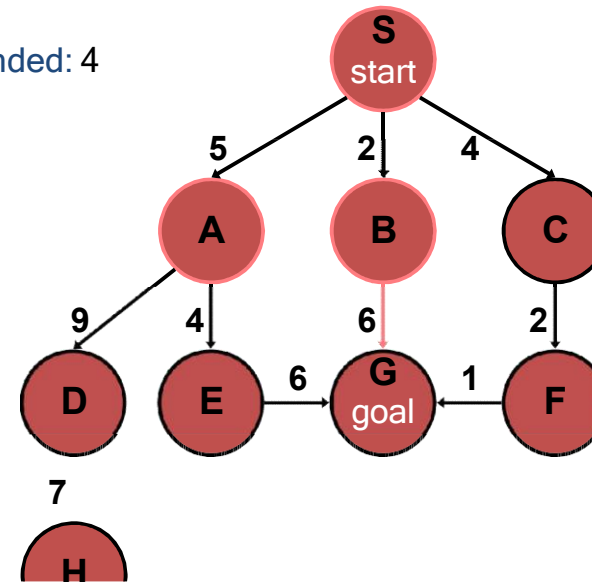
Iterative-Deepening Search(IDS)



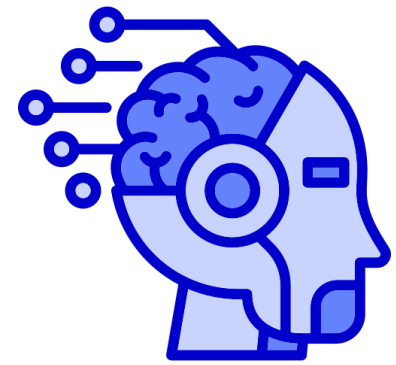
deepeningSearch (problem)

depth: 2, # of nodes tested: 6(3), expanded: 4

expnd. node	Frontier
	{S}
S	{A,B,C}
A.	{B,C}
B.	{C}
C.	{}
S	{A,B,C}
A	{D,E,B,C}
D.	{E,B,C}
E.	{B,C}
B no test	{G,C}



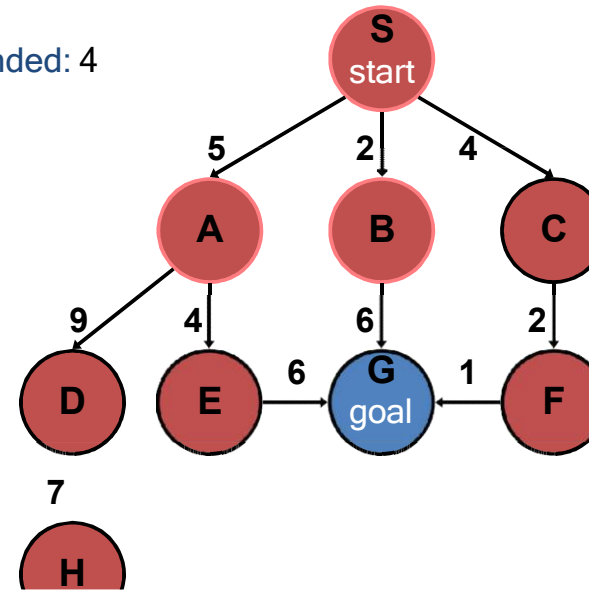
Iterative-Deepening Search(IDS)



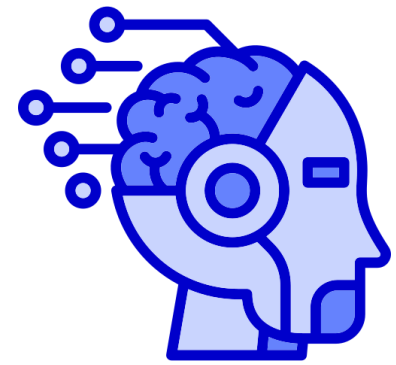
deepeningSearch (problem)

depth: 2, # of nodes tested: 7(3), expanded: 4

expnd. node	Frontier
	{S}
S	{A,B,C}
A	{B,C}
B.	{C}
C.	{}
S	{A,B,C}
A	{D,E,B,C}
D.	{E,B,C}
E.	{B,C}
B	{G,C}
G goal	{C} no expand



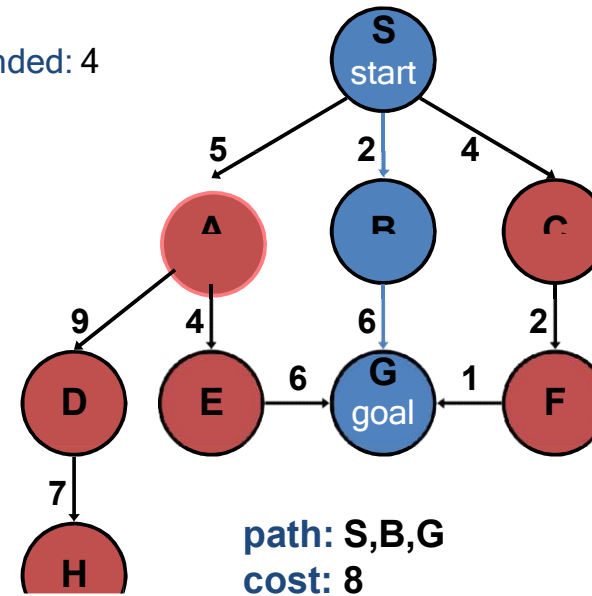
Iterative-Deepening Search(IDS)

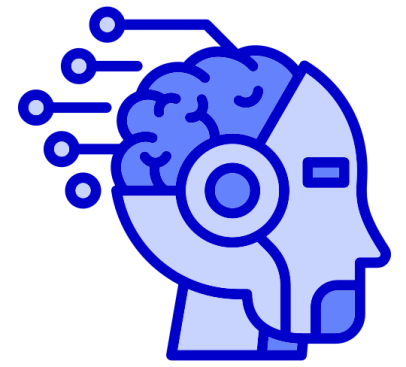


deepeningSearch (problem)

depth: 2, # of nodes tested: 7(3), expanded: 4

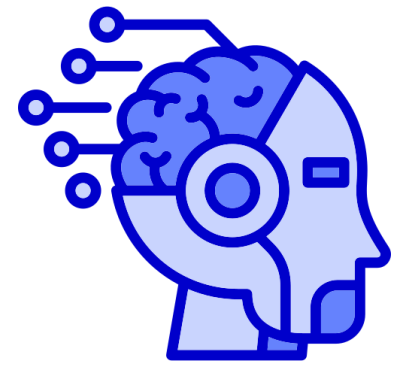
expnd. node	Frontier
	{S}
S	{A,B,C}
A.	{B,C}
B.	{C}
C.	{ }
S	{A,B,C}
A	{D,E,B,C}
D.	{E,B,C}
E.	{B,C}
B	{G,C}
G	{C}





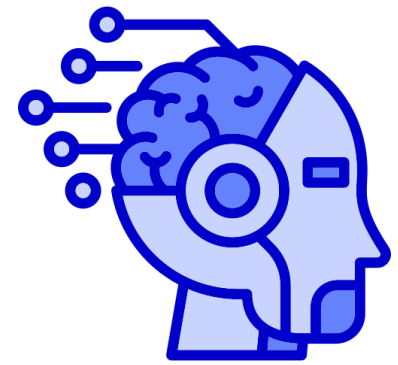
Iterative-Deepening Search(IDS)

- Has advantages of BFS
 - completeness
 - optimality as stated for BFS
- Has advantages of DFS
 - limited space
 - in practice, even with the redundant effort still finds longer paths more quickly than BFS



Iterative-Deepening Search(IDS)

- Space complexity: $O(bd)$ (i.e., linear like DFS)
- Time complexity is a little worse than BFS or DFS
 - because nodes near the top of the search tree are generated multiple times (redundant effort)
- Worst case time complexity: $O(b^d)$ exponential
 - because most nodes are near the bottom of tree

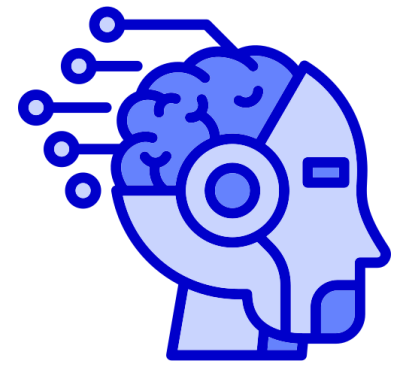


Iterative-Deepening Search(IDS)

How much redundant effort is done?

- The number of times the nodes are generated:
$$1b^d + 2b^{(d-1)} + \dots + db \leq b^d / (1 - 1/b)^2$$

$$= O(b^d)$$
 - d : the solution's depth
 - b : the branching factor at each non-leaf node
- For example: $b = 4$
$$4^d / (1 - 1/4)^2 = 4^d / (.75)^2 = 1.78 \times 4^d$$
 - in the worst case, 78% more nodes are searched (redundant effort) than exist at depth d
 - as d increases, this % decreases



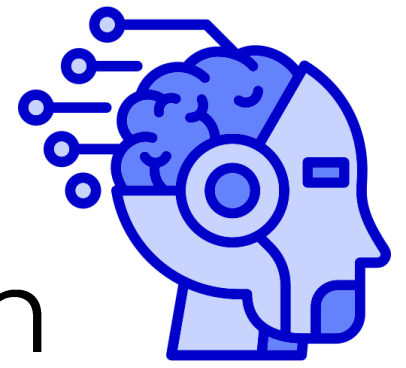
Iterative-Deepening Search

- **Trades a little time for a huge reduction in space**
 - lets you do breadth-first search with
(more space efficient) depth-first search

applications, e.g., games

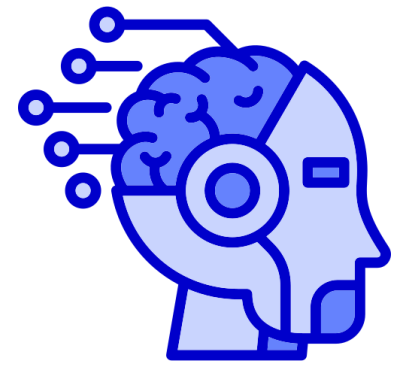
Anytime algorithm: good for response-time critical

- An “anytime” algorithm is an [algorithm](#) that can return a valid solution to a [problem](#) even if it's interrupted at any time before it ends. The algorithm is expected to find better and better solutions the more time it keeps running.



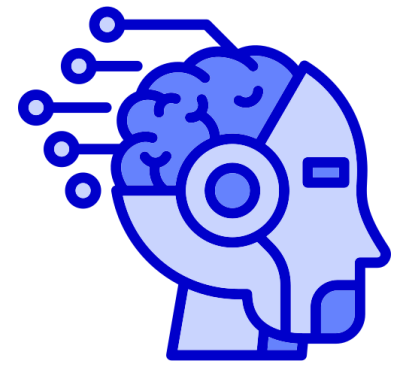
Iterative Deepening Search

- It's a **Depth First Search**, but it does it one level at a time, gradually increasing the limit, until a goal is found.
- Combine the benefits of depth-first and breadth-first search
- like DFS, modest memory requirements $O(bd)$
- like BFS, it is complete when branching factor is finite, and optimal when the path cost is a nondecreasing function of the depth of the node



Iterative Deepening Search

- May seem wasteful because states are generated multiple times
- but actually not very costly, because nodes at the bottom level are generated only once
- In practice, however, the overhead of these multiple expansions is small, because most of the nodes are towards leaves (bottom) of the search tree:
thus, the nodes that are evaluated several times (towards top of tree) are in relatively small number.
- Iterative deepening is the preferred uninformed search method when the search space is large and the depth of the solution is unknown



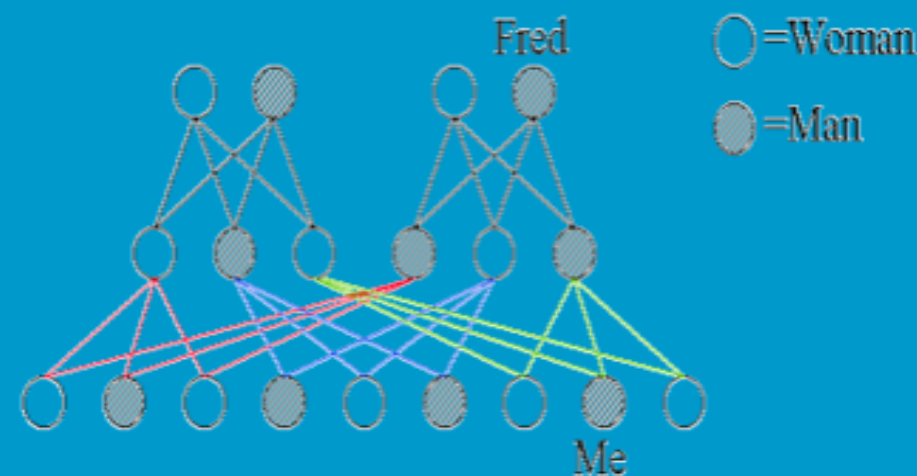
Properties of Iterative Deepening Search

Combines the best of breadth-first and depth-first search strategies.

- Completeness: Yes,
- Time complexity: $O(b^d (1) + (b) + (b+b^2) + (b+b^2+b^3) + \dots + (b^d))$
- Space complexity: $O(b^d)$
- Optimality: Yes, if step cost = 1

Direction of Search

- Sometimes, the direction of the search will make a difference to the size of the search. It may be that there are less nodes to consider if you search back from the goal node up to the start node.
- A search algorithm attempting to find out if I am a descendant of Fred can take two different routes.
- It can start at Fred and work its way down, or it can start at me and work its way up. All I am interested is in finding a route between Fred and myself.
- So let's look at a top to bottom search, starting with Fred and attempting to find me.



- Going down the tree we see that there are three nodes to check at level one.
- From these three, there are a further nine nodes to check, so we may have to check up to 12 nodes, excluding Fred.
- If we go up the tree starting at Me, we see that one level up there are two nodes to examine, my parents.
- Going on from my parents, there are four more nodes - my grandparents of whom Fred is one.
- Going up the tree I only have to consider 6 nodes as opposed to 12.
- If John wants to find out if he is a descendant of Ethel, who is not two but 5 generations away the difference becomes even more noticeable.
- The top-down approach would have to consider up to 363 nodes whereas the bottom-up approach would only have to consider 62 at the most - a significant difference.

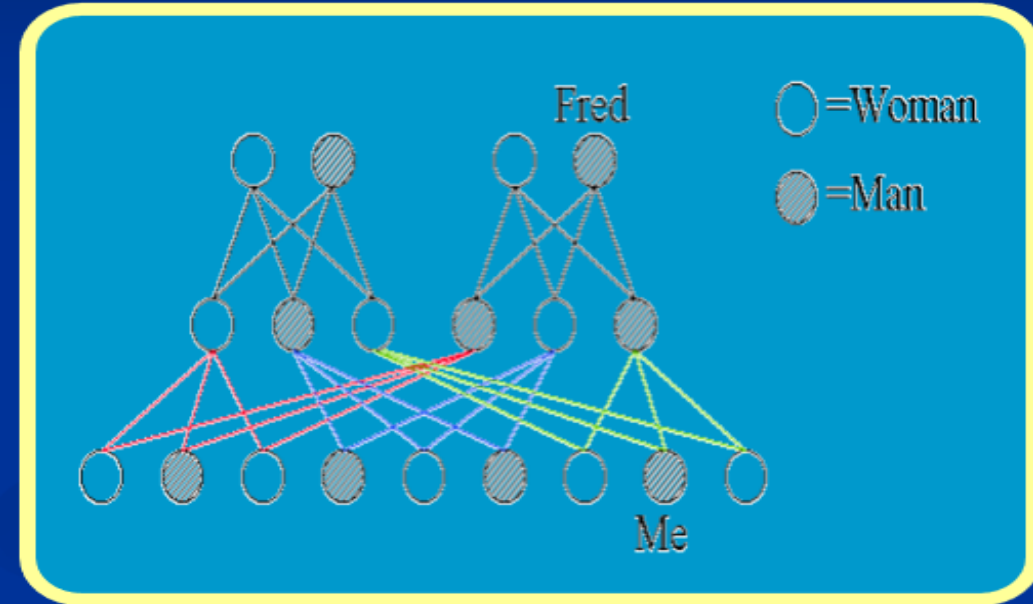
Direction of Search (continued)...

■ Why is this?

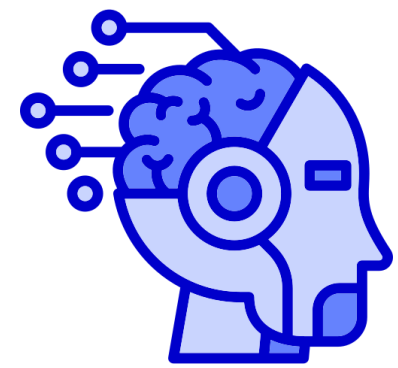
■ Well it's all in the branching factor,

- for the top-down approach the branching factor is 3 - each couple has three children -so at level n there are 3^n nodes to consider.
- The branching factor the other way is 2 as each person has only two parents, thus n levels up the tree from the start there are 2^n nodes to consider.

■ So if you have a tree with one average branching factor one way and a smaller average the other way, and if you are free to solve the problem in either direction, it is probably advisable to try solving it going in the direction that gives the least branching factor.



Comparison of Uninformed Searches



Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes ^a	Yes ^{a,b}	No	No	Yes ^a	Yes ^{a,d}
Time	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^l)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(bl)$	$O(bd)$	$O(b^{d/2})$
Optimal?	Yes ^c	Yes	No	No	Yes ^c	Yes ^{c,d}

Figure 3.21 Evaluation of tree-search strategies. b is the branching factor; d is the depth of the shallowest solution; m is the maximum depth of the search tree; l is the depth limit. Superscript caveats are as follows: ^a complete if b is finite; ^b complete if step costs $\geq \epsilon$ for positive ϵ ; ^c optimal if step costs are all identical; ^d if both directions use breadth-first search.